

Design and Implementation of an Arbitrary Precision Arithmetic Library in Java

Author:

Rishit Mittal

CS24BTECH11053

April 24, 2025

Abstract

This report presents the design rationale, implementation strategy, verification approach, and key insights gained during the development of a Java-based library for arbitrary precision arithmetic. The library enables mathematical operations on integers and floating-point numbers with theoretically unbounded precision, making it suitable for scientific and cryptographic computations requiring high accuracy.

Contents

1	Introduction	2
2	Features and Language Extensions	2
3	User Guide (README Summary)	3
4	Git Commit History Snapshot	3
5	Limitations	4
6	Verification and Testing Approach	4
7	Key Learnings	4

1 Introduction

This project aims to implement a high-precision arithmetic library capable of operating on both integers and floating-point numbers with arbitrary digit length. It was developed with object-oriented programming principles and is compatible with both Java and Python interfaces. The motivation behind this project stems from the limitations of built-in data types in traditional programming languages which cannot natively support operations on extremely large or precise numerical values.

2 Features and Language Extensions

This section outlines the major features provided by the library and presents illustrative examples for each. These enhancements effectively extend the capabilities of the Java language in terms of numeric computation.

Key Functionalities

- Support for arbitrary-length integer arithmetic via the `AInteger` class
- Floating-point arithmetic with up to 1000 digits after the decimal point using `AFloat`
- Support for basic arithmetic operations: addition, subtraction, multiplication, and division
- Operator overloading through intuitive method interfaces (e.g., `add()`, `sub()`, etc.)
- Dual CLI support through both Java and Python scripts
- Modular and reusable packaging using `.jar`, with Maven integration

Illustrative Examples

Integer Addition:

```
AInteger a = new AInteger("1234");
AInteger b = new AInteger("4321");
AInteger result = a.add(b); // Output: 5555
System.out.println(result);
```

Floating-point Multiplication (via Python CLI):

```
$ python run.py float mul 1.5 2.5
# Output: 3.75
```

Design Decisions

[To be completed by the team: Include internal representation, precision handling, rounding methods, parsing strategies, etc.]

3 User Guide (README Summary)

This section summarizes the README for end-users, outlining how to compile, execute, and integrate the library in different environments.

Compilation and Execution (Java CLI)

```
javac src/main/java/arbitraryarithmetic/*.java src/main/java/MyInfArith.java
java -cp src/main/java MyInfArith int add 123 456
```

Python CLI Interface

```
python run.py float mul 1.5 2.5
```

Using the Packaged JAR

```
java -cp target/aarithmetic.jar MyInfArith int sub 1000 250
```

Library Integration in Java Projects

```
import arbitraryarithmetic.*;

public class TestMain {
    public static void main(String[] args) {
        AInteger a = new AInteger("1234");
        AInteger b = new AInteger("4321");
        AInteger result = a.add(b);
        System.out.println(result); // Should print 5555
    }
}
```

Building with Maven

```
mvn clean package
```

4 Git Commit History Snapshot

Include a visual summary of the git commit timeline. This helps in understanding the evolution and collaborative effort behind the project.

Figure 1: Snapshot of Git Commit History

5 Limitations

Despite the library's wide utility, it has the following limitations:

- *[Add known performance issues or unsupported edge cases]*
- *[Mention absence of advanced mathematical functions if applicable]*
- *[Discuss possible overflow or rounding issues in float handling if any]*

6 Verification and Testing Approach

Robust verification was carried out through manual and automated test cases designed to cover a comprehensive set of scenarios.

Test Scenarios

- Comparisons of positive vs. negative numbers
- Handling of extremely large numerical inputs
- Behavior with zero and identity elements
- Consistency in floating-point division and rounding
- Graceful handling of malformed or invalid inputs

Example Test Case

Input: `AInteger("999999999999") + AInteger("1")`

Expected Output: `AInteger("1000000000000")`

[Mention any automated testing frameworks or scripts used.]

7 Key Learnings

This section reflects on the technical and collaborative takeaways from the project.

- *[Discuss conceptual learning in numerical computing or Java internals]*
- *[Highlight any optimizations or data structure decisions that were particularly valuable]*
- *[Mention challenges in parsing, memory handling, or CLI integration]*